

## UNIT IV: Angular Js & jQuery

---

### Q1. Define MVC architecture with respect to AngularJS.

MVC stands for **Model-View-Controller**. It is a software design pattern used to separate the application's logic into three interconnected parts. This separation makes the application easier to manage, test, and update.

In the context of AngularJS, the parts are:

1. **Model:** This is the data of the application. It is the "source of truth." In AngularJS, the Model is typically plain JavaScript objects. It does not contain any business logic; it just holds the data (e.g., a user's name, a list of items).
2. **View:** This is the user interface (UI) or what the user sees. In AngularJS, the View is the HTML template. It uses AngularJS directives (like `ng-model` or `{{ }}`) to display the data from the Model.
3. **Controller:** This is the "logic" part that connects the Model and the View. In AngularJS, the Controller is a JavaScript function. Its job is to:
  - Set up the initial data (the Model).
  - Provide functions and logic that the View can use (e.g., what happens when a user clicks a button).
  - It uses the **\$scope** object to pass data (Model) and functions from the Controller to the View.

#### How they work together:

- The **Controller** prepares the data (Model) and puts it on the **\$scope**.
- The **View** (HTML) binds to this data on the **\$scope**.
- When the Model changes (e.g., in the Controller), the View updates automatically.
- When the user changes data in the View (e.g., in a text box), the Model updates automatically (this is called two-way data binding).

---

### Q2. Explain AngularJS Controllers with example.

An AngularJS **Controller** is a JavaScript constructor function that controls a specific part of the HTML page (the View). It is responsible for managing the data (Model) and logic for that part of the view.

#### Key Roles of a Controller:

- It initializes the **\$scope** object.
- It adds data (properties) and functions (behavior) to the **\$scope**.
- The View then binds to the properties and functions on the **\$scope** to display data and execute logic.

A controller is defined using the `.controller()` method on a module.

**Example:** Let's create a simple app that has a controller to store a user's name and a function to greet them.

#### 1. The JavaScript (app.js):

```
// 1. Create the module
```

```

var myApp = angular.module('myApp', []);

// 2. Create the controller
myApp.controller('myCtrl', function($scope) {
  // 3. Add data (Model) to the $scope
  $scope.name = 'Ajit';

  // 4. Add a function (behavior) to the $scope
  $scope.sayHello = function() {
    $scope.greeting = 'Hello, ' + $scope.name + '!';
  };
});

```

## 2. The HTML (index.html):

```

<!DOCTYPE html>
<html ng-app="myApp"> <head>
  <script src="angular.min.js"></script>
  <script src="app.js"></script>
</head>
<body ng-controller="myCtrl"> Enter your name:
  <input type="text" ng-model="name"> <button ng-click="sayHello()">Greet Me</button>
  <h2>{{ greeting }}</h2> </body>
</html>

```

In this example, myCtrl controls the <body> tag. It sets \$scope.name to 'Ajit'. The ng-model directive creates a two-way binding with \$scope.name. When the user clicks the button, the ng-click directive calls the \$scope.sayHello() function, which updates the \$scope.greeting property, and the <h2> tag displays it.

### Q3. Explain AngularJS Modules with example.

An AngularJS **Module** is the main container for an AngularJS application. It is used to group related components, such as controllers, services, directives, and filters.

Think of a module as the "main entry point" of the application. It helps in:

- **Organization:** Keeps the code clean and separated by feature.
- **Dependency Management:** A module can declare its dependencies on other modules.
- **Preventing Global Variables:** It creates a "namespace" for your application parts, so you don't pollute the global scope.

You define a module using the angular.module() function.

#### Example:

1. Defining a Module (app.js): The angular.module() function takes two arguments:

1. The name of the module (e.g., 'myApp').
2. An array of dependencies (other modules this module needs). If it has no dependencies, you must still provide an empty array [].

```

// This defines a module named 'myApp' with no dependencies.
var myApp = angular.module('myApp', []);

```

2. Attaching a Controller to the Module (app.js): Once the module is defined, you can chain methods like `.controller()` to it.

```
myApp.controller('mainController', function($scope) {  
  $scope.message = 'This is my application module!';  
});
```

3. Using the Module in HTML (index.html): You attach the module to your HTML using the `ng-app` directive.

```
<!DOCTYPE html>  
<html ng-app="myApp">  
  <head>  
    <script src="angular.min.js"></script>  
    <script src="app.js"></script>  
  </head>  
  
  <body ng-controller="mainController">  
    <h1>{{ message }}</h1>  
  </body>  
</html>
```

This bootstraps the application, telling Angular that the `<html>` element and its children belong to the `myApp` module.

---

#### **Q4. What are directives? Name some of the most commonly used directives in AngularJS application.**

**Directives** are the most important feature of AngularJS. They are markers on a DOM element (like an attribute, tag name, or CSS class) that tell AngularJS's HTML compiler to attach a specific behavior to that element or even transform the element and its children.

In simple terms, directives are what "teach" HTML new tricks. They extend HTML's syntax. Most directives in AngularJS start with the prefix `ng-`.

##### **Most Commonly Used Directives:**

1. **ng-app**: Initializes (bootstraps) an AngularJS application. It tells Angular which part of the page is the "root" of the application.
  - **Example:** `<html ng-app="myApp">`
2. **ng-model**: Provides **two-way data binding**. It binds the value of an HTML control (like `<input>`, `<select>`, `<textarea>`) to a variable on the `$scope`.
  - **Example:** `<input type="text" ng-model="userName">`
3. **ng-controller**: Attaches a controller function to a specific part of the DOM (the View).
  - **Example:** `<div ng-controller="myCtrl">...</div>`
4. **ng-bind** (or `{{ }}` expressions): Provides **one-way data binding**. It binds the content of an HTML element to a variable on the `$scope`. The `{{ }}` "expression" syntax is a shortcut for this.
  - **Example:** `<p ng-bind="message"></p>` or `<p>{{ message }}</p>`
5. **ng-repeat**: Loops through a collection (like an array) and repeats a block of HTML for each item in the collection.

- **Example:** `<li ng-repeat="item in items">{{ item.name }}</li>`
  - 6. **ng-click:** Executes a function on the \$scope when an element is clicked.
    - **Example:** `<button ng-click="saveData()">Save</button>`
  - 7. **ng-show / ng-hide:** Shows or hides an HTML element based on a boolean (true/false) expression. It works by adding or removing the `display: none;` CSS property.
    - **Example:** `<div ng-show="isLoggedIn">Welcome user!</div>`
  - 8. **ng-if:** Similar to ng-show, but it completely **removes** the element from the DOM if the expression is false, and **re-creates** it if it becomes true. This is more powerful but more resource-intensive.
    - **Example:** `<div ng-if="isAdmin">Admin Panel</div>`
- 

## Q6. What is data binding in AngularJS?

**Data binding** is the automatic synchronization of data between the Model and the View. It is a core feature of AngularJS that greatly simplifies development by eliminating the need to write code to manually update the UI or listen for changes. In AngularJS, data binding is primarily "two-way."

**1. One-Way Data Binding (Model to View):** This is when data from the Model (e.g., `$scope.name` in the controller) is displayed in the View (HTML). If the data in the Model changes, the View updates automatically.

- This is achieved using directives like **ng-bind** or, more commonly, the expression syntax `{{ }}`.
  - *Controller:* `$scope.message = 'Hello';`
  - *View:* `<p>{{ message }}</p>` (Displays "Hello")
  - If the controller later changes `$scope.message = 'Hi'`, the `<p>` tag will automatically update to show "Hi".

**2. Two-Way Data Binding (Model <=> View):** This is the most powerful feature. It creates a live, two-way connection between the Model and the View. If the Model changes, the View updates. If the View (user) changes the data (e.g., typing in a text box), the Model updates automatically.

- This is achieved using the **ng-model** directive, which is used on form input elements.
  - *Controller:* `$scope.username = 'Tejashree';`
  - *View:* `<input type="text" ng-model="username">`
  - *Result:* The input box will initially show "Tejashree". If the user types in the box and changes it to "Ajit", the `$scope.username` variable in the controller will instantly become "Ajit" without any extra code.

This "two-way" binding saves developers from writing a lot of "glue code" to connect the UI with the data.

---

## Q7. List Core features of Angular JS with diagram.

AngularJS is a powerful JavaScript framework for building dynamic web applications. Its core features are:

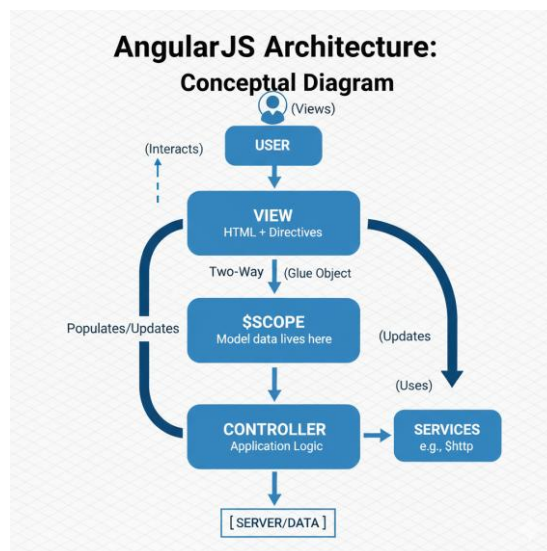
1. **MVC (Model-View-Controller) Architecture:** Separates the application logic (Controller), data (Model), and UI (View). This makes the code organized and easy to maintain.
2. **Two-Way Data Binding:** The automatic synchronization of data between the Model and View. When the Model changes, the View updates, and vice-versa.
3. **Directives:** Custom HTML attributes and tags (like `ng-model`, `ng-repeat`) that extend HTML's

capabilities and add dynamic behavior.

4. **Dependency Injection (DI):** A design pattern where components (like controllers and services) have their dependencies "injected" into them, rather than creating them. This makes code modular, reusable, and easy to test. For example, the `$scope` and `$http` objects are *injected* into controllers.
5. **Modules:** Containers that group related parts of an application (controllers, services, etc.) to keep the code organized.
6. **Services:** Reusable, singleton objects that are used for shared logic or data. AngularJS has many built-in services (like `$http` for making API calls) and you can create your own.
7. **Filters:** Used to format data for display in the View (e.g., `{{ myDate | date:'short' }}` or `{{ price | currency }}`).
8. **Templates:** The View (HTML) which contains HTML, directives, and data-binding expressions.

### Diagram (Text-Based Description):

A conceptual diagram of AngularJS would show the following flow:



- The **User** interacts with the **View**.
- The **View** (e.g., an input with `ng-model`) updates the **\$scope** (Model).
- The **Controller** can also update the **\$scope**.
- When the **\$scope** (Model) changes, it automatically updates the **View**.
- The **Controller** uses **Services** (like `$http`) to fetch data from a **Server**, and then puts that data on the **\$scope** to be displayed.

### Q8. List various parts of Angular JS.

An AngularJS application is built using several key parts or "building blocks" that work together:

1. **Modules:** The main containers for the application (defined with `angular.module()`).
2. **Controllers:** JavaScript functions that provide logic and data to a View (defined with `.controller()`).
3. **Views (Templates):** The HTML that the user sees, containing directives and expressions.
4. **\$scope:** The "glue" object that links the Controller and the View. It holds the Model data and functions.
5. **Directives:** Custom attributes/tags that add behavior to HTML (e.g., `ng-app`, `ng-model`, `ng-repeat`).
6. **Expressions (`{{ }}`):** The code, similar to JavaScript, placed inside double curly braces in the View to display data (e.g., `{{ 5 + 5 }}` or `{{ user.name }}`).

7. **Filters:** Functions used within expressions or directives to format data before it is displayed (e.g., | uppercase, | currency).
  8. **Services:** Reusable, singleton objects used for shared tasks like fetching data from a server (\$http), logging (\$log), or custom application logic.
  9. **Dependency Injector (DI):** The underlying system that manages and provides all the parts (like services and \$scope) to the functions (like controllers) that need them.
- 

### Q9. Explain any three filters with example.

**Filters** in AngularJS are used to format the data that is displayed in the View. They are used within expression bindings {{ }} or directives using the "pipe" character |.

**1. uppercase / lowercase Filter:** Converts a text string to all uppercase or all lowercase.

*Controller:* \$scope.studentName = "Ajit Mali";

*View:*

```
<p>Original: {{ studentName }}</p>
<p>Uppercase: {{ studentName | uppercase }}</p>
<p>Lowercase: {{ studentName | lowercase }}</p>
```

*Output:*

```
Original: Ajit Mali
Uppercase: AJIT MALI
Lowercase: ajit mali
```

**2. currency Filter:** Formats a number as a currency. By default, it uses the dollar sign (\$) and two decimal places, but the symbol can be changed.

*Controller:* \$scope.bookPrice = 450.75;

*View:*

```
<p>Price (Default): {{ bookPrice | currency }}</p>
<p>Price (Rupees): {{ bookPrice | currency : "₹" }}</p>
```

*Output:*

```
Price (Default): $450.75
Price (Rupees): ₹450.75
```

**3. filter Filter (for arrays):** This is a powerful filter used with ng-repeat to search and filter an array. It selects a subset of items from an array based on a search string.

*Controller:*

```
$scope.students = [
  { name: 'Rahul', city: 'Pune' },
  { name: 'Amit', city: 'Mumbai' },
  { name: 'Tejashree', city: 'Pune' }
];
```

*View:*

Search for a student:

```
<input type="text" ng-model="searchQuery">
<ul>
  <li ng-repeat="student in students | filter:searchQuery">
    {{ student.name }} ({{ student.city }})
  </li>
</ul>
```

Output:

If the user types "Pune" into the search box, the list will automatically update to show only:

- Rahul (Pune)
- Tejashree (Pune)

---

### Q10. What are the benefits of AngularJS?

AngularJS (the first version) provides several benefits for building dynamic, single-page applications (SPAs):

1. **Simplified Development (Two-Way Data Binding):** The automatic data binding between the Model and View dramatically reduces the amount of code needed. Developers don't have to write manual code to update the UI when data changes.
2. **MVC Architecture:** It promotes a clean separation of concerns (data, logic, and presentation). This makes the code more organized, easier to maintain, and simpler to debug.
3. **Testability:** AngularJS was designed with testing in mind. Features like Dependency Injection make it easy to "mock" (simulate) dependencies and write unit tests for individual components (like controllers) in isolation.
4. **Reusable Components (Directives):** Directives allow developers to create their own custom, reusable HTML elements and attributes. This is powerful for building a library of UI components (like a special calendar or a data grid).
5. **Declarative UI:** The View (HTML) *declares* what the UI should look like and how it should be bound to the data, rather than using imperative code (like jQuery) to describe *how* to build the UI step-by-step. This makes the templates more readable.
6. **Dependency Injection (DI):** DI makes components more modular. A controller doesn't need to know how to create the \$http service; it just "asks" for it, and Angular's injector provides it. This makes code cleaner and more flexible.
7. **Faster Prototyping:** You can get a data-driven web application up and running very quickly due to features like data binding and directives.

---

### Q11. What are different forms of form events?

In AngularJS, "form events" are not just traditional JavaScript events (like onsubmit). Instead, AngularJS enhances standard HTML forms and provides a set of properties (or states) that track the form's condition and user interaction.

These "states" can be used to show or hide error messages, disable buttons, and apply validation styling. The main states are available on the form itself (e.g., myForm) and on individual input controls (e.g., myForm.emailInput):

#### 1. Form and Control States:



- **\$pristine** (boolean): The user has **not** yet interacted with or modified the form or control.
  - **Opposite:** \$dirty (The user *has* changed the value).
- **\$untouched** (boolean): The form or control has **not** been "blurred" (i.e., the user has not clicked into it and then clicked out).
  - **Opposite:** \$touched (The user *has* blurred the control).
- **\$valid** (boolean): The form or control's value meets all its validation requirements (e.g., required, type="email").
  - **Opposite:** \$invalid (The value is *not* valid).

**2. The \$error Object:** This is an object on the control that contains detailed information about *which* validation rules have failed.

- **Example:** If an input is required and also needs to be an email, its \$error object might look like this:
  - myForm.emailInput.\$error.required = true; (if it's empty)
  - myForm.emailInput.\$error.email = true; (if it's not a valid email format)

**How these states are used like events:** You can use these states with directives like ng-show or ng-disabled to create an interactive form.

```
<form name="myForm">
  <label>Email:</label>
  <input type="email" name="emailInput" ng-model="data.email" required>

  <div ng-show="myForm.emailInput.$touched && myForm.emailInput.$invalid">
    <span ng-show="myForm.emailInput.$error.required">
      Email is required.
    </span>
    <span ng-show="myForm.emailInput.$error.email">
      That is not a valid email.
    </span>
  </div>

  <button type="submit" ng-disabled="myForm.$invalid">
    Submit
  </button>
</form>
```

## Q12. Explain the concept of JQuery with selecting, creating, appending elements.

**Concept of jQuery:** jQuery is a fast, small, and feature-rich JavaScript library. Its motto is "Write Less, Do More." It is not a new language; it is a file full of pre-written JavaScript functions that simplify common and complex tasks, especially:

- HTML DOM (Document Object Model) manipulation.
- Event handling.
- Ajax requests (Asynchronous JavaScript and XML).
- Animations and effects.

The main advantage of jQuery is that it makes it very easy to find, select, and manipulate HTML elements on a page, and it handles cross-browser differences.



The core of jQuery is the **\$** function (which is a shortcut for `jQuery()`).

**1. Selecting Elements:** jQuery uses CSS-style selectors inside the `$()` function to find (or "select") HTML elements in the DOM.

- **Select by ID:** `$('#myId')` - `var title = $('#main-title');` (Selects the element with `id="main-title"`)
- **Select by Class:** `$('.myClass')` - `var allButtons = $('.btn');` (Selects *all* elements with `class="btn"`)
- **Select by Tag Name:** `$('tagName')` - `var allParagraphs = $('p');` (Selects *all* `<p>` elements)

**2. Creating Elements:** You can create a new HTML element from scratch by passing a string of HTML to the `$()` function.

```
// This creates a new <p> element in memory (it's not on the page yet)
var newParagraph = $('<p>This is a brand new paragraph!</p>');

// You can also add attributes
var newButton = $('<button id="btn-click" class="btn">Click Me</button>');
```

**3. Appending Elements:** Once you have selected a container and created a new element, you can add (append) the new element to the container.

- **.append():** Adds the new element as the *last child* inside the selected element.
- **.prepend():** Adds the new element as the *first child* inside the selected element.
- **.after():** Adds the new element *after* (as a sibling) the selected element.
- **.before():** Adds the new element *before* (as a sibling) the selected element.

```
<div id="container">
  <h2>Original Title</h2>
</div>
<script>
  // 1. Select the container
  var container = $('#container');

  // 2. Create a new element
  var newText = $('<p>We added this text with jQuery!</p>');

  // 3. Append the new element inside the container
  container.append(newText);
</script>
```

**Resulting HTML:**

```
<div id="container">
  <h2>Original Title</h2>
  <p>We added this text with jQuery!</p>
</div>
```

---

### Q13. What are the selectors in jQuery? How many types of selectors in jQuery?

jQuery selectors are the most important part of the jQuery library. They are patterns (strings) used to find and select HTML elements from the DOM (Document Object Model). Once an element is selected, you can perform actions on it (like hiding it, changing its content, or adding an event listener). jQuery's selector syntax is based on the selectors from CSS (Cascading Style Sheets).

**Types of Selectors in jQuery:** There are many selectors, but they fall into a few main types:

1. **Basic Selectors:**

- **Element Selector ('tag'):** Selects all elements with the specified tag name.
  - *Example:* \$('p') selects all <p> elements.
- **ID Selector ('#id'):** Selects a single element with a specific id. This is the fastest selector.
  - *Example:* \$('#myDiv') selects <div id="myDiv">.
- **Class Selector ('.class'):** Selects all elements with a specific class.
  - *Example:* \$('.highlight') selects all elements with class="highlight".
- **Universal Selector ('\*'):** Selects all elements on the page. (Use rarely, it's slow).

2. **Hierarchy (or Relational) Selectors:**

- **Descendant Selector ('ancestor descendant'):** Selects all elements that are descendants (children, grandchildren, etc.) of a given ancestor.
  - *Example:* \$('div p') selects all <p> elements that are inside a <div>.
- **Child Selector ('parent > child'):** Selects all elements that are *direct* children of a given parent.
  - *Example:* \$('ul > li') selects <li> elements that are direct children of a <ul>, but not <li> elements inside a nested <ul>.
- **Next Sibling Selector ('prev + next'):** Selects the one element that is *immediately* after the prev element.
  - *Example:* ('h2 + p') selects the first <p> tag that comes right after an <h2>.

3. **Attribute Selectors:**

- Selects elements based on the presence or value of their attributes.
- *Example:* \$('input[type="text"]') selects all <input> elements with type="text".
- *Example:* \$('a[href="www.google.com"]') selects links with a specific href.

4. **Filter Selectors (or Basic Filters):**

- These selectors filter a set of elements, often starting with a colon (:).
- *Example:* \$('li:first') selects the first <li> element.
- *Example:* \$('li:last') selects the last <li> element.
- *Example:* \$('tr:even') selects the even-numbered table rows (0, 2, 4...).

---

## Q14. What is event delegation in jQuery?

**Event delegation** is a technique for handling events more efficiently. Instead of attaching a separate event handler to *every single child element* (which can be slow and use a lot of memory), you attach **one single event handler** to a *parent* element.

**How it works (Event Bubbling):** When an event (like a click) happens on an element (e.g., an <li>), that event "bubbles up" the DOM tree. It goes from the <li> to its parent <ul>, then to the <body>, then to the <html>. With event delegation, the parent element (e.g., the <ul>) *listens* for the event as it bubbles up.

**When it catches the event, it checks two things:**

1. Did the event happen on a child element I care about (e.g., an <li>)?
2. If yes, then execute the function.

**How to do it in jQuery:**

We use the .on() method with three arguments:

```
$(parent-selector).on(eventName, 'child-selector', function() { ... });
```

```
<ul id="myList">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
  ...
  <li>Item 100</li>
</ul>
```

### Bad Way (100 event handlers):

```
$('#myList li').on('click', function() {
  $(this).css('color', 'red');
});
```

### Good Way (Event Delegation - 1 event handler):

```
$('#myList').on('click', 'li', function() {
  // 'this' still refers to the <li> that was clicked
  $(this).css('color', 'red');
});
```

### Advantages of Event Delegation:

1. **Performance:** You only have *one* event handler in memory instead of hundreds or thousands. This is much faster.
2. **Works for Dynamic Elements:** This is the most important benefit. If you add *new* <li> elements to the list later using JavaScript, the event handler on the parent <ul> will **automatically work for them**. The "Bad Way" would not work for new elements.

---

## Q15. What are the jQuery effects?

**jQuery effects** are pre-built functions that allow you to create visual animations and transitions on HTML elements. These functions make it easy to show, hide, or animate elements to make a web page more dynamic and engaging.

The most common jQuery effects can be grouped into categories:

### 1. Basic Show/Hide Effects:

- These methods change the display property of elements.
- **.hide(speed, callback):** Hides the selected element(s).
  - **.show(speed, callback):** Shows the selected element(s).
  - **.toggle(speed, callback):** Toggles between showing and hiding the element(s).
    - speed can be a string ("slow", "fast") or a number in milliseconds (e.g., 1000 for 1 second).
    - callback is a function to run *after* the effect is complete.
    - *Example:* `$('#myDiv').hide('slow');`

### 2. Fading Effects:

- These methods change the opacity of elements.
- **.fadeIn(speed, callback):** Fades in the element (makes it visible).
  - **.fadeOut(speed, callback):** Fades out the element (makes it invisible).
  - **.fadeToggle(speed, callback):** Toggles between fading in and out.
  - **.fadeTo(speed, opacity, callback):** Fades the element to a specific opacity level (e.g., 0.5 for

50%).

- Example: `$('#myButton').fadeOut(500);`

**3. Sliding Effects:** These methods change the height of elements.

- **.slideDown(speed, callback):** Slides the element down (reveals it).
- **.slideUp(speed, callback):** Slides the element up (hides it).
- **.slideToggle(speed, callback):** Toggles between sliding up and down.
  - Example: `$('.menu').slideToggle('fast');`

**4. Custom Animation (.animate()):** This is the most powerful effect. It allows you to create custom animations by changing any numeric CSS property.

- **Syntax:** `.animate({ cssProperties }, speed, callback)`

// This will move the box 250px to the right, make it 50% transparent, and increase its height all at the same time, over 1 second.

```
$('#myBox').animate({  
  left: '250px',  
  opacity: 0.5,  
  height: '+=150px' // increase height by 150px  
}, 1000);
```

---

## Q16. What are the advantages of jQuery?

jQuery became the most popular JavaScript library in the world because it solved many problems for web developers. Its main advantages are:

1. **Cross-Browser Compatibility:** This was its biggest advantage. In the past, browsers like Internet Explorer, Firefox, and Chrome all handled JavaScript differently. Developers had to write complex code to check which browser the user had. jQuery solves this by providing a single, consistent API that works the same way across all major browsers.
2. **Simplicity ("Write Less, Do More"):** jQuery simplifies complex JavaScript tasks. A task that might take 10 lines of regular JavaScript (like an Ajax call or a complex animation) can often be done in a single line of jQuery.
3. **Powerful Selectors:** It provides a very easy and powerful way to find (select) any HTML element on the page using CSS-style syntax.
4. **Easy DOM Manipulation:** Once an element is selected, jQuery makes it simple to change its content (`.html()`, `.text()`), change its attributes (`.attr()`), or change its style (`.css()`).
5. **Simplified Event Handling:** It provides a simple and consistent way to handle user events like click, hover, submit, etc. Features like event delegation (`.on()`) are very powerful.
6. **Built-in Effects and Animations:** jQuery comes with a set of easy-to-use functions for animations (`.fadeIn()`, `.slideToggle()`, `.animate()`) which make websites more interactive.
7. **Simplified Ajax:** jQuery's Ajax methods (like `$.ajax()`, `$.get()`, `$.post()`) make it much easier to load data from a server in the background without reloading the entire page.
8. **Large Community and Plugins:** Because it is so popular, there is a massive community and a huge number of free "plugins" that you can add to your site for extra functionality (like image carousels, date pickers, etc.).

---

## Q17. Explain JQuery in detail.

jQuery is a fast, lightweight, and feature-rich JavaScript Library. It is not a programming language itself, but rather a large, single file of pre-written JavaScript code (jquery.min.js). Its famous motto is "Write Less, Do More."

The primary purpose of jQuery is to make it much easier to use JavaScript on your website. It simplifies common but complex tasks like:

- Finding and selecting HTML elements (DOM traversal).
- Modifying HTML elements (DOM manipulation).
- Handling user events (like clicks and key presses).
- Creating animations and visual effects.
- Communicating with a server (Ajax).

To use jQuery, you must first include the library file in your HTML, usually in the <head> or just before the closing </body> tag.

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
<script src="js/jquery.min.js"></script>
```

**The Document Ready Event:** Because jQuery is used to manipulate HTML, we must wait for the HTML document to be fully loaded before we run any jQuery code. We do this by wrapping our code in a "document ready" function:

```
$(document).ready(function() {
    // All jQuery code goes in here.
});
// A common shortcut for the same thing:
$(function() {
    // All jQuery code goes in here.
});
```

### Core Features Explained:

1. **The jQuery Object (\$):** The \$ symbol is the heart of jQuery. It is an alias for the jQuery function. This function is used for almost everything, especially selecting elements.
2. **Selectors (Finding Elements):** jQuery uses CSS-style selectors to find elements.
  - \$('#myId'): Selects by ID.
  - \$('.myClass'): Selects by class.
  - \$('p'): Selects by tag name.
  - \$('div p'): Selects all <p> tags inside <div> tags.
3. **DOM Manipulation (Changing the Page):** Once you select an element, you can easily change it.
  - .html("..."): Changes the inner HTML of an element.
  - .text("..."): Changes the inner text (safer).
  - .val("..."): Gets or sets the value of a form input.
  - .attr('href', '...'): Gets or sets an attribute.
  - .css('color', 'red'): Changes a CSS property.
  - .append(...) / .prepend(...): Adds new elements.
4. **Event Handling (Responding to Users):** jQuery provides a simple way to respond to user actions.
  - \$('#myButton').on('click', function() { ... });
  - This is the modern way to handle events. It can also be used for event delegation, which is very efficient.
  - Common events: click, mouseenter (hover), mouseleave, keydown, submit.
5. **Effects and Animations:** jQuery has built-in functions to create simple animations.

- .hide(), .show(), .toggle()
  - .fadeIn(), .fadeOut(), .fadeToggle()
  - .slideUp(), .slideDown(), .slideToggle()
  - .animate(): For custom animations (e.g., moving an element).
6. **Ajax (Asynchronous JavaScript and XML):** Ajax allows a web page to request data from a server in the background without reloading the page. jQuery's Ajax functions are much simpler than plain JavaScript.
- \$.ajax(): The main, powerful function.
  - \$.get(): A shortcut for simple GET requests.
  - \$.post(): A shortcut for simple POST requests.

jQuery was revolutionary because it solved the major problem of cross-browser incompatibility. It provided a single, simple API that worked everywhere, allowing developers to build complex, interactive websites much faster.

### Q18. What are events in jQuery?

**Events** are actions that happen in the browser, which the program can detect and respond to. Most events are user-generated, but they can also be triggered by the browser itself.

In jQuery, "events" refer to the simplified methods used to "listen" for these actions and execute a function (called an "event handler" or "callback") when the action occurs.

#### Examples of Common Events:

- **Mouse Events:**
  - click: The user clicks an element.
  - dblclick: The user double-clicks an element.
  - mouseenter: The user's mouse pointer moves onto an element (hover).
  - mouseleave: The user's mouse pointer moves off of an element.
  - mousemove: The mouse is moved while over an element.
- **Keyboard Events:**
  - keydown: The user presses a key down.
  - keyup: The user releases a key.
  - keypress: The user presses a character key (deprecated, keydown is preferred).
- **Form Events:**
  - submit: The user submits a form (e.g., by clicking a "submit" button).
  - change: The value of a form element (like <input>, <select>) is changed.
  - focus: The user clicks into or tabs into a form element.
  - blur: The user clicks out of or tabs out of a form element.
- **Document/Window Events:**
  - ready: The DOM (HTML structure) is fully loaded and ready to be manipulated.
  - load: The entire page, including all images and resources, is fully loaded.
  - scroll: The user scrolls the page.

**How to Handle Events in jQuery:** The modern and most flexible way to handle any event is with the **.on()** method.

**Syntax:** `$( 'selector' ).on( 'eventName', function() { ... } );`

This code will select the element with id="myButton" and "listen" for a click event. When the click

happens, it will run the function, which hides all <p> tags.

```
// 1. Wait for the document to be ready
$(document).ready(function() {
  // 2. Select the button
  $('#myButton').on('click', function() {
    // 3. This function runs when the button is clicked
    alert('Button was clicked!');
    $('p').hide(); // Hides all paragraphs
  });
});
```

---

## Q19. What is an attribute in jQuery?

An **attribute** is a piece of information that provides extra details about an HTML element. Attributes are always written inside the element's opening tag and usually come in name/value pairs.

### Examples of HTML Attributes:

- In <a href="page.html">Link</a>, href is the attribute.
- In , src and alt are attributes.
- In <input type="text" id="name" disabled>, type, id, and disabled are attributes.

**Working with Attributes in jQuery:** jQuery provides simple methods to "get" (read) and "set" (write) the values of these attributes.

**1. Get an Attribute's Value: .attr():** To read the value of an attribute, you pass one argument (the attribute name) to the .attr() method.

```
// HTML: <a id="myLink" href="google.com">Google</a>
var linkUrl = $('#myLink').attr('href');
// linkUrl is now "google.com"
```

**2. Set an Attribute's Value: .attr():** To change or set the value of an attribute, you pass two arguments: the attribute name and the new value.

```
// HTML: 
// Change the 'src' attribute
$('#myImage').attr('src', 'new.jpg');
// Also change the 'alt' attribute
$('#myImage').attr('alt', 'This is the new image');
// Resulting HTML: 
```

**3. Remove an Attribute: .removeAttr():** To completely remove an attribute from an element.

```
// HTML: <input type="text" disabled>
$('#myInput').removeAttr('disabled');
// Resulting HTML: <input type="text">
```

**Note: .prop() vs .attr():** For attributes that represent "properties" (boolean values like checked, selected, disabled), it is better to use the .prop() method.

- .attr() gets the *default* value (from the HTML).
  - .prop() gets the *current* value (e.g., if a user has checked a checkbox).
  - *Example:* Use \$('#myCheckbox').prop('checked', true); to check a box.
-



## Q20. What is jQuery UI?

**jQuery UI** is an official companion library for jQuery. It is a curated set of user interface (UI) **interactions, effects, widgets, and themes** built on top of the core jQuery library. While core jQuery is for DOM manipulation and Ajax, **jQuery UI is for building rich, interactive, and "stateful" user interfaces**. You must include *both* the jQuery library *and* the jQuery UI library in your page to use it.

### Key Features of jQuery UI:

1. **Widgets:** These are pre-built, complex UI components that would be very difficult to build from scratch. You just call a single function to create them.
  - **datepicker():** Adds a pop-up calendar to an input field.
  - **tabs():** Creates a tabbed content area.
  - **accordion():** Creates collapsible content panels.
  - **dialog():** Turns any <div> into a draggable modal pop-up window.
  - **autocomplete():** Provides a "search suggestions" dropdown for an input.
  - **slider()** and **progressbar()**.
2. **Interactions:** These are functions that add complex behaviors to any existing HTML element.
  - **draggable():** Makes an element movable by dragging it with the mouse.
  - **droppable():** Makes an element a "target" for draggable elements.
  - **resizable():** Lets the user resize an element by dragging its corner.
  - **selectable():** Lets the user select multiple items from a list.
  - **sortable():** Lets the user re-order items in a list by dragging them.
3. **Effects:** jQuery UI extends the core jQuery effects library with more advanced animations, like color-based animations and "class" animations.
  - *Examples:* `.effect('shake')`, `.effect('explode')`, `.effect('bounce')`.
4. **ThemeRoller:** jQuery UI comes with a powerful tool called ThemeRoller. It is a web-based application that lets you visually design a custom theme (colors, fonts, corners) for all the widgets, and then download the custom CSS file. This ensures all your UI components have a consistent look and feel.

---

## Q21. Is jQuery a JavaScript or JSON library file?

jQuery is a **JavaScript library file**.

### Explanation:

- **JavaScript (JS):** jQuery is a library *written in* JavaScript. The file you include in your page (e.g., `jquery.min.js`) is a single, large file containing thousands of lines of JavaScript code. This code defines all the helpful functions (like `$()`, `.on()`, `.animate()`, etc.) that make up the jQuery library.
- **JSON (JavaScript Object Notation):** JSON is a **data format**. It is a lightweight, text-based way to store and transport data. It is *not* a library or a programming language. It is just a syntax for organizing data, similar to XML but simpler.

**The Relationship:** jQuery (a JavaScript library) can work with JSON (a data format).

**For example,** jQuery's Ajax functions are often used to fetch data from a server. In modern web applications, the server usually sends that data formatted as JSON. jQuery has built-in methods (like `$.getJSON()`) that can automatically fetch and parse this JSON data, turning it into JavaScript objects that you can easily use in your code.

---